
Rapport d'avancement de stage

Semaine 7

Integration : une requete unifiee pour l'interpretation ancree
(transcription + regles RAG + situation spatiale) vers un JSON exploitable

Un copilote d'intelligence artificielle pour le controle aerien

Auteur Nicolas Marano
Projet Simulateur de controleur aerien (PoC, 10 semaines)
Etablissement Universite de Reims Champagne-Ardenne (URCA)
Calcul (IA) Supercalculateur ROMEO : NVIDIA GH200 (Grace-Hopper, aarch64)
Domaine Intelligence artificielle & controle du trafic aerien
Periode Semaine 7 du stage

Resume. Cette semaine marque le debut de l'integration des modules developpes separement. Une *requete unifiee* est construite : elle assemble en une seule entree la transcription de la clairance, les regles de phraseologie recuperees par le RAG et une situation spatiale fixe (la topologie du secteur). Le LLM produit en sortie un JSON conforme decrivant l'action a mener (`{callsign, action, value}`). Une chaine de validation deterministe garantit que ce JSON reste propre et exploitable : bornes physiques, fixes connus du secteur, coherence semantique de l'instruction et presence de l'aeronef sur le radar.

Table des matières

1	Contexte et positionnement de la semaine	1
1.1	Synthese chiffree de la semaine	1
2	Semaine 7 : La requete unifiee	1
2.1	Les trois entrees de la requete	1
2.2	La situation spatiale fixe : le graphe de secteur	2
2.3	Construction de la requete unifiee	2
2.4	Orchestration : de la transcription au JSON	3
2.5	Garantir un JSON propre et exploitable	3
2.6	Resultats et mesures	4
3	Bilan et transition	4

1 Contexte et positionnement de la semaine

Le projet automatise le dialogue radio pilote / controleur a l'interieur d'un simulateur de trafic aerien. Une instruction parlée est transcrite, ancree dans la phraseologie OACI, transformee en une commande validee, executee dans le simulateur BlueSky, puis restituee en voix de pilote synthetisee. La chaine cible se resume ainsi :

voix pilote (VHF) → Whisper STT → NER + LLM avec RAG (OACI Doc 4444) → JSON valide
→ simulateur BlueSky (manoeuvre avion) → voix de readback synthetisee → boucle

Les semaines precedentes ont produit, et valide separement, le module de **perception** (semaine 4), le module d'**interpretation** par RAG (semaine 5) et le module de **restitution vocale** (semaine 6). La semaine 7 ouvre la phase d'**integration**. L'objectif est de reunir les entrees du module d'interpretation au sein d'une **requete unifiee** : plutot que de traiter la transcription isolement, le systeme la combine avec les regles de phraseologie pertinentes et avec une representation de l'espace aerien, afin que le LLM dispose de tout le contexte necessaire pour produire, en une passe, un JSON propre decrivant l'action. A ce stade, la situation spatiale fournie est **fixe** (la topologie statique du secteur) ; l'injection de l'etat de trafic en temps reel constitue l'etape suivante.

1.1 Synthese chiffree de la semaine

Indicateur	Resultat
Entrees combinees dans la requete unifiee	3 (transcription, regles, espace)
Fixes de la situation spatiale (secteur)	7
Segments / separation imposee	6 / 5 NM
Plus court chemin ENTRY_W → EXIT_E	101 NM
JSON exploitable produit (40 transcriptions reelles)	100 %
Couches de validation du JSON de sortie	4

Table 1 – Vue d'ensemble des resultats obtenus durant la semaine 7.

2 Semaine 7 : La requete unifiee

Objectif de la semaine

Construire une requete unifiee prenant en entree la transcription, les regles du RAG et une situation spatiale fixe, et produisant en sortie un JSON propre et exploitable decrivant l'action a mener (`{callsign, action, value, [wpt]}`).

2.1 Les trois entrees de la requete

La requete unifiee agrege trois sources d'information complementaires :

- **La transcription** : le texte de la clairance, issu du module de reconnaissance vocale (semaine 4) ou saisi directement.
- **Les regles du RAG** : les fiches de phraseologie les plus pertinentes pour cette clairance, recuperees par recherche vectorielle (semaine 5, $k = 4$).
- **La situation spatiale fixe** : une representation de l'espace aerien (le graphe de secteur), qui fournit au LLM la liste des points connus et la topologie, et qui sert ensuite a valider les ordres de routage.

2.2 La situation spatiale fixe : le graphe de secteur

La situation spatiale est decrite par un graphe statique du secteur, charge depuis `secteur_graphe.json`. Il comporte 7 points (de type *entry*, *fix* ou *exit*), 6 segments orientes avec leur distance, et une separation imposee de 5 NM (figure 1). La classe `SectorGraph` expose la liste des fixes, le voisinage, le plus court chemin (Dijkstra sur les distances) et un resume textuel compact de la topologie. C'est ce resume qui est injecte dans la requete pour donner au LLM une conscience spatiale, sans alourdir le *prompt*.

Listing 1 – Resume textuel de la topologie injecte dans la requete (`topology_text()`).

```
Sector (separation 5.0 NM).
Fixes: ENTRY_W(entry), BALMO(fix), CROSS(fix), DELTA(fix), EXIT_E(exit), ENTRY_S(entry),
       NORTH(exit).
Segments: ENTRY_W-BALMO 25.0NM; BALMO-CROSS 25.5NM; CROSS-DELTA 25.5NM; CROSS-ENTRY_S 25.0NM;
          CROSS-NORTH 25.0NM; DELTA-EXIT_E 25.0NM.
```

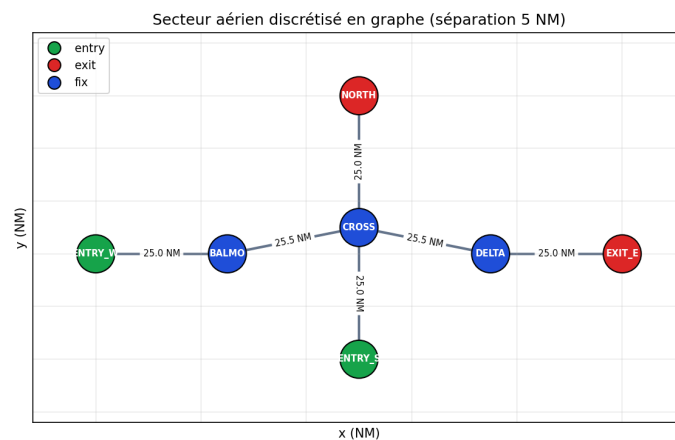


Figure 1 – Situation spatiale fixe : le graphe du secteur (points d'entree, fixes, sorties ; segments et distances). Il fournit le contexte spatial de la requete et valide les ordres de routage. A ce stade, cette topologie est statique (sans etat de trafic).

2.3 Construction de la requete unifiee

La fonction `build_messages` assemble les trois entrees en un unique message adresse au LLM, precede d'un *prompt* systeme strict qui impose une sortie au format JSON et seulement cela. Les regles recuperees, la topologie du secteur, les indices d'entites (indicatif, intentions) et la transcription sont concatenes dans un ordre fixe.

Listing 2 – Assemblage de la requete unifiee (extrait de `atc_llm.py`).

```
rules = "\n".join(f"- {d['title']}: {d['text']}" for d, _ in retrieved)
sector = f"Sector graph: {graph.topology_text()}\n\n"
hint = f"callsign={ner_callsign}; intentions_detectees={ner_intentions}"
user = (f"Applicable ICAO phraseology rules:\n{rules}\n\n"
        f"{sector}"
        f"NER hints: {hint}\n\n"
        f"Controller transcription: \"{transcription}\"\n\n"
        "Return ONLY the JSON array of orders.")
messages = [{"role": "system", "content": SYSTEM}, {"role": "user", "content": user}]
```

Sur un exemple, la requete assemblee et la sortie attendue prennent la forme suivante :

Listing 3 – Requete unifiee assemblee (abregee) et JSON produit par le LLM.

```
Applicable ICAO phraseology rules:
- Instruction de cap (heading): ... action HDG, value = cap en degres ...

Sector graph: Sector (separation 5.0 NM). Fixes: ENTRY_W(entry), BALMO(fix), ...

NER hints: callsign=AFR1234; intentions_detectees=['turn']

Controller transcription: "air france one two three four turn right heading two seven zero"

Return ONLY the JSON array of orders.
--> [{"callsign": "AFR1234", "action": "HDG", "value": 270}]
```

2.4 Orchestration : de la transcription au JSON

La fonction `interpret()` enchaîne les étapes de la requete unifiee : extraction des entites (NER) pour l'indicatif et les intentions, recuperation des $k = 4$ fiches pertinentes, assemblage de la requete, generation par le LLM (Mistral 7B), puis analyse tolerante du JSON et validation. La sortie est une liste d'ordres structures, chacun associe a sa traduction executable ou a un motif de rejet.

2.5 Garantir un JSON propre et exploitable

L'integration ne se limite pas a produire du JSON : elle doit produire du JSON *exploitable sans risque*. Quatre verifications deterministes s'appliquent en aval de la generation.

1. **Schema et bornes** : chaque ordre est traduit par `json_to_trafscript`; toute valeur hors bornes (HDG 0–360, ALT 0–45000, SPD 0–350) ou action inconnue est rejetee.
2. **Coherence spatiale** : un ordre de routage (ADDWPT) doit cibler un fix *connu* du secteur, sinon il est rejete ou normalise vers le fix correct.
3. **Coherence semantique** : si la clairance dit “descend” (sans “climb”) mais que l'ordre vise un niveau *au-dessus* du niveau actuel de l'aeronef, l'ordre est juge incoherent (transcription tronquee ou hallucination) et rejete plutot qu'execute; le critere est symetrique pour “climb”.
4. **Realisme radio** : un ordre adresse a un indicatif absent du radar ne produit aucune reponse et n'est pas transmis au simulateur.

Contraintes techniques et choix d'ingenierie

A ce stade de l'integration, la situation spatiale fournie au LLM est volontairement *statique* : seule la topologie du secteur est injectee, pas encore l'etat de trafic (positions et niveaux des aeronefs), qui fera l'objet de l'etape suivante. La topologie est transmise sous forme d'un resume textuel compact plutot que du JSON complet du graphe, afin de limiter la taille de la requete et de ne pas diluer l'instruction. Le *prompt* systeme reste strict pour que l'ajout du bloc spatial ne pousse pas le modele a commenter sa reponse : seule la liste JSON est demandee, et une analyse tolerante recupere le tableau meme si le modele ajoute du texte. La verification de coherence semantique requiert l'altitude courante de l'aeronef ; elle s'applique donc au moment de l'execution, lorsque cet etat est disponible.

2.6 Resultats et mesures

Resultats cles

- **Requete unifiee** operationnelle : transcription, regles RAG et situation spatiale fixe combinees en une seule passe vers le LLM.
- **JSON exploitable** produit sur 40 transcriptions reelles : 100 %.
- **Validation de routage** : un ADDWPT vers un fix connu (BALMO) est accepte ; vers un point inconnu (*tango bravo*), il est rejete.
- **Garde-fou semantique** : une clairance tronquee de type "descend flight level one" qui ferait monter l'aeronef est detectee et rejete.

Analyse. La requete unifiee atteint son objectif : sur les transcriptions reelles evaluees, la sortie est systematiquement un JSON analysable, et les quatre couches de validation eliminant les ordres dangereux, mal route ou semantiquement incoherents avant toute execution. L'apport de l'integration ne tient pas seulement a l'ajout du contexte spatial, mais aussi a la *convergence* des verifications en un point unique : un meme ordre est confronte aux bornes physiques, a la topologie du secteur et a la coherence de l'instruction. La principale limite a ce stade est que la conscience spatiale du LLM reste statique : il connait la geometrie du secteur mais pas la position courante des aeronefs. C'est cette derniere information qui permettra, ensuite, des decisions reellement situees (anticipation de conflit, routage tenant compte du trafic).

3 Bilan et transition

La semaine 7 etablit le socle de l'integration : une requete unifiee qui combine la transcription, les regles du RAG et une situation spatiale fixe, et dont la sortie est un JSON propre et exploitable decrivant l'action a mener. La fiabilite de cette sortie repose sur quatre verifications deterministes appliquees en aval du LLM (bornes, coherence spatiale, coherence semantique, presence au radar).

Prochaines etapes. L'etape directe consiste a remplacer la situation spatiale *fixe* par l'etat de trafic *courant* (positions, caps et niveaux des aeronefs lus dans le simulateur), afin de passer d'une conscience spatiale statique a une conscience situee. Suivront la consolidation de la boucle temps reel de bout en bout et une evaluation quantitative sur des scenarios complets.